BrainAI

# ProSuite Industrial

API & WRAPPER SUITE

**PRODUCT DOCUMENTATION**

BrainAI (Inhaber: Sascha Alexander Köhne)

| BrainAI Inhaber: Sascha Alexander Köhne | PROSUITE INDUSTRIAL | 🟢 PUBLIC CLASSIFICATION |
|---|---|---|
| Core Architecture & Development | **ProKey Integration Guide** | Access Level: Public / Beta |

---

## Revision History

| Version | Date | Author | Status | Description |
|---|---|---|---|---|
| 1.0.0 | 2025-01-01 | S. Köhne | Stable | Initial Release of ProKey Wrapper Documentation |
| 1.1.0 | 2026-02-27 | S. Köhne | Stable | Full API Integration Guide (C#, Java, Node.js, Python, VB.NET) |

## 1 Product Overview

The **ProKey Wrapper Suite** serves as the high-level language interface to the native ProKey Hardware Entropy Engine. Unlike standard random number generators found in respective programming languages (such as `java.util.Random` or `Math.random()`), which merely provide deterministic pseudo-random numbers (PRNG), these wrappers tap into true physical entropy.

This is absolutely mandatory for generating asymmetric key pairs, AES master keys, and highly secure session tokens in enterprise environments.

**Supported Core Features:**

- **Native Performance:** Direct memory access without unnecessary copy operations (Zero-Copy approach).
- **Hardware RNG:** Access to the optimized C routines `ProKey_Generate` and `ProKey_Fill256Bit`.
- **Platform Agnosticism:** Automatic detection and binding of Linux (`.so`) or Windows (`.dll`) libraries.

## 2 Common Fundamentals & API Structure

All wrappers abstract the native C interface and offer a standardized set of functions and enumerations.

**Core Calls:**

1. **`GenerateBytes(int length)`:** Requests an arbitrary amount of entropy and writes it into an allocated byte array.
2. **`GenerateKey(ProKeyBits bits)`:** Type-safe generation based on industrial standard lengths (128, 256, 512, 1024 bits).
3. **`Generate256BitKey()`:** A performance wrapper that internally calls the optimized native function `ProKey_Fill256Bit` (Ideal for AES-256).

## 3 C# (.NET) Integration

The C# wrapper utilizes `P/Invoke` (`DllImport`) to interface with the native C library in unmanaged memory.

- **Memory Management:** Arrays are allocated as `byte[]` and automatically passed to the C environment and written back by the marshaller.
- **Implementation Example:**

```
using ProChat.Core.Crypto;

// Generating an AES-256 key
byte[] masterKey = ProKey.Generate256BitKey();

// Generating a 512-bit session token
byte[] sessionToken = ProKey.GenerateKey(ProKeyBits.Bit512);
```

## 4 Java (JNA) Integration

The Java binding is established via **Java Native Access (JNA)**, eliminating the need for complex JNI C code (Java Native Interface).

- **Type Safety:** Uses `NativeSize` for correct cross-platform handling of `size_t` (32-bit vs. 64-bit pointers).
- **Implementation Example:**

```
import com.brainai.prokey.ProKey;

// Retrieving 16 bytes (128 bits) for an Initialization Vector (IV)
byte[] iv = ProKey.generateBytes(16);

// Conversion to hex for transmission
String hexToken = ProKey.bytesToHex(ProKey.generateKey(ProKey.ProKeyB
```

## 5 JavaScript (Node.js) Integration

Utilizes `ffi-napi` and native V8 `Buffer` objects. This is the most performant way to process data streams in Node.js backends, as the buffers reside directly in the C++ memory of the V8 engine.

- **Use Case:** Backend services (Express.js/NestJS), JWT signature keys.
- **Implementation Example:**

```
const { ProKey, ProKeyBits } = require('./prokey');

// Generate buffer directly and convert to hex
const aesKeyBuffer = ProKey.generate256BitKey();
const hexKey = aesKeyBuffer.toString('hex').toUpperCase();
```

## 6 Python Integration

The Python wrapper relies on the system's built-in `ctypes` module and requires no external dependencies (like cffi).

- **Security Aspect:** The return value is strictly provided as an immutable `bytes` object to prevent accidental modifications of the key in memory.
- **Implementation Example:**

```
from prokey import ProKey, ProKeyBits

# Generate nonce
nonce = ProKey.generate_bytes(12)

# 512-bit Key
token = ProKey.generate_key(ProKeyBits.BIT_512)
```

## 7 VB.NET Integration

Specifically designed for legacy and OT (Operational Technology) systems. Provides a type-safe class that embeds seamlessly into older Windows Forms or industrial control applications.

- **Implementation Example:**

```
Imports ProChat.Core.Crypto

Dim myAesKey As Byte() = ProKey.Generate256BitKey()
Console.WriteLine("AES Key: " & ProKey.BytesToHex(myAesKey))
```

## 8 Best Practices & Security (Zero-Trust)

- **Garbage Collection (GC) Warning:** In high-level languages (like Java or C#), the garbage collector cleans up memory asynchronously. As soon as you no longer need a key, you should manually overwrite the byte array with zeros if possible (`Array.Clear()` in .NET or `Arrays.fill(key, (byte) 0)` in Java) before dropping the reference. This minimizes the attack window for RAM dumps.
- **Hex Conversion:** Never transmit binary entropy directly over text protocols (like JSON/HTTP). Always use the hex or Base64 conversion functions provided in the wrappers or your specific programming language.
- **Hardware Limitations:** True hardware entropy is a finite resource (physical generation rate). Use ProKey for master keys, salts, and session tokens. For massive, extremely fast data streams (e.g., hard drive wiping), you should use a CSPRNG (Cryptographically Secure PRNG) initially seeded with ProKey.

---

| | | |
|---|---|---|
| **© 2026 Sascha Köhne** | ✉️ **Contact:** [koehne83@googlemail.com](mailto:koehne83@googlemail.com) | 🛡️ **ProTU-Verified** |

| | | |
|---|---|---|
| CEO / System Architect | BrainAI Inhaber : Sascha Alexander Köhne | We don't need **BRUTEFORCE**, we know **PHYSICS** |